CSE 1242 - COMPUTER PROGRAMMING II
Non-Graded Programming Assignment # 3

In this assignment, you should implement the following programs in C.

1. Write a function that will take one integer pointer, `*number`, and an integer `N`. Then reverse number's last N digits.

- The function header must be as the following:

```
void reverseN(int *number, int N)
```

- You should take input numbers from the user in `main` function and then invoke the function **`reverseN`** with appropriate parameters.
- The **`main`** function should print the result, as the updated value of the number.

Sample Runs:

Run 1:

```
176 2
167
```

Run 2:

```
63712 3
63217
```

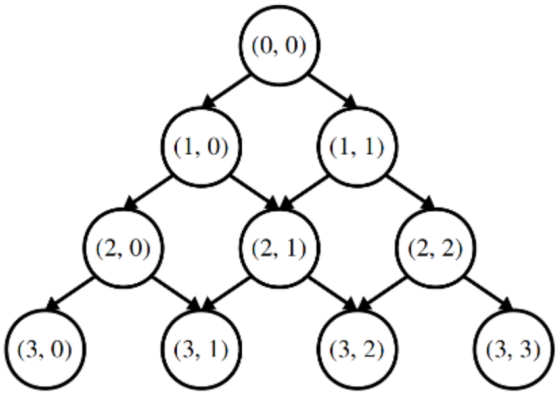
Run 2:

```
32145 5
54123
```

Run 2:

```
345 5
N must be less than 4!
```

2. Suppose that we will build a pyramid with young people for the celebration of May 19th Commemoration of Atatürk, Youth and Sports Day.
The pyramid will look as follows:



Here, each circle shows the column and row ids of each person.

- Suppose that the column and row index of the person at top (in the row 0) is (0, 0). (1, 0) shows the id of the person in the 1st row column 0, etc.
- Suppose that each person weighs exactly 80 kilograms.
- So, the person in the first row (0, 0) has no weight on his/her back.
- However, the person at index (1,0) has 40 kilos (half the weight of the previous person on his/her shoulders) on his/her back.
- Likewise, the person at index (1, 1) has 40 kilos on his/her back because she/he shares half of the person at index (0, 0).
- The people at index (2, 0) and (2, 2) have extra 60 kilos on their backs (the people on the first row (1,0) and (1, 1) have 80+40 kilos individually).
- However, the person at index (2, 1) has 120 kilos on his/her back because he/she weighs the half of the people at (1, 0) and (1, 1).
- In short, each person has extra half weight of the people on his/her shoulders.

Your task is to implement the following function:

```
double calculateWeightBack(int row, int col)
```

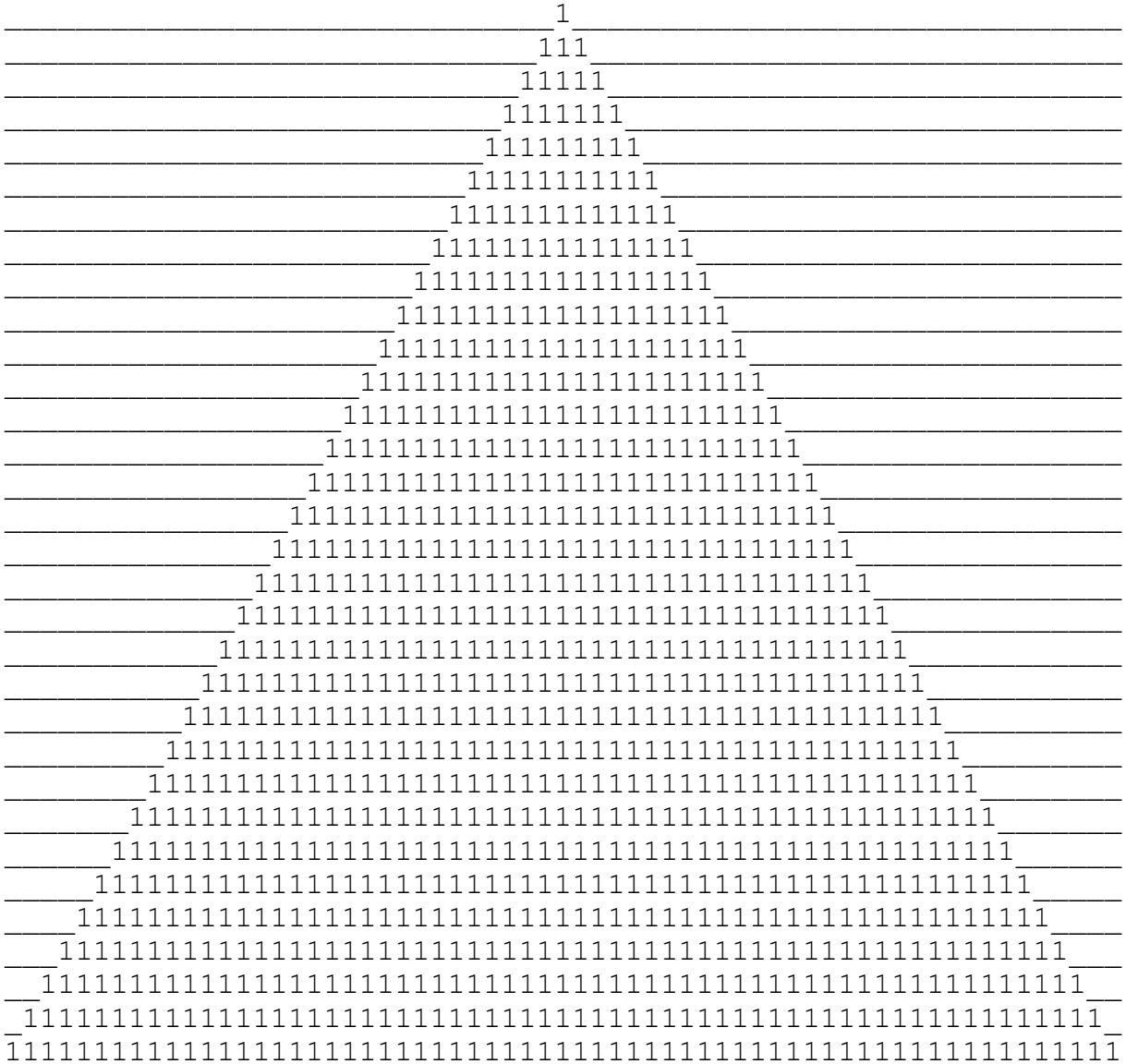
that takes the row and column index of a person and returns the total weight on his/her back.

Please implement the function in two ways:

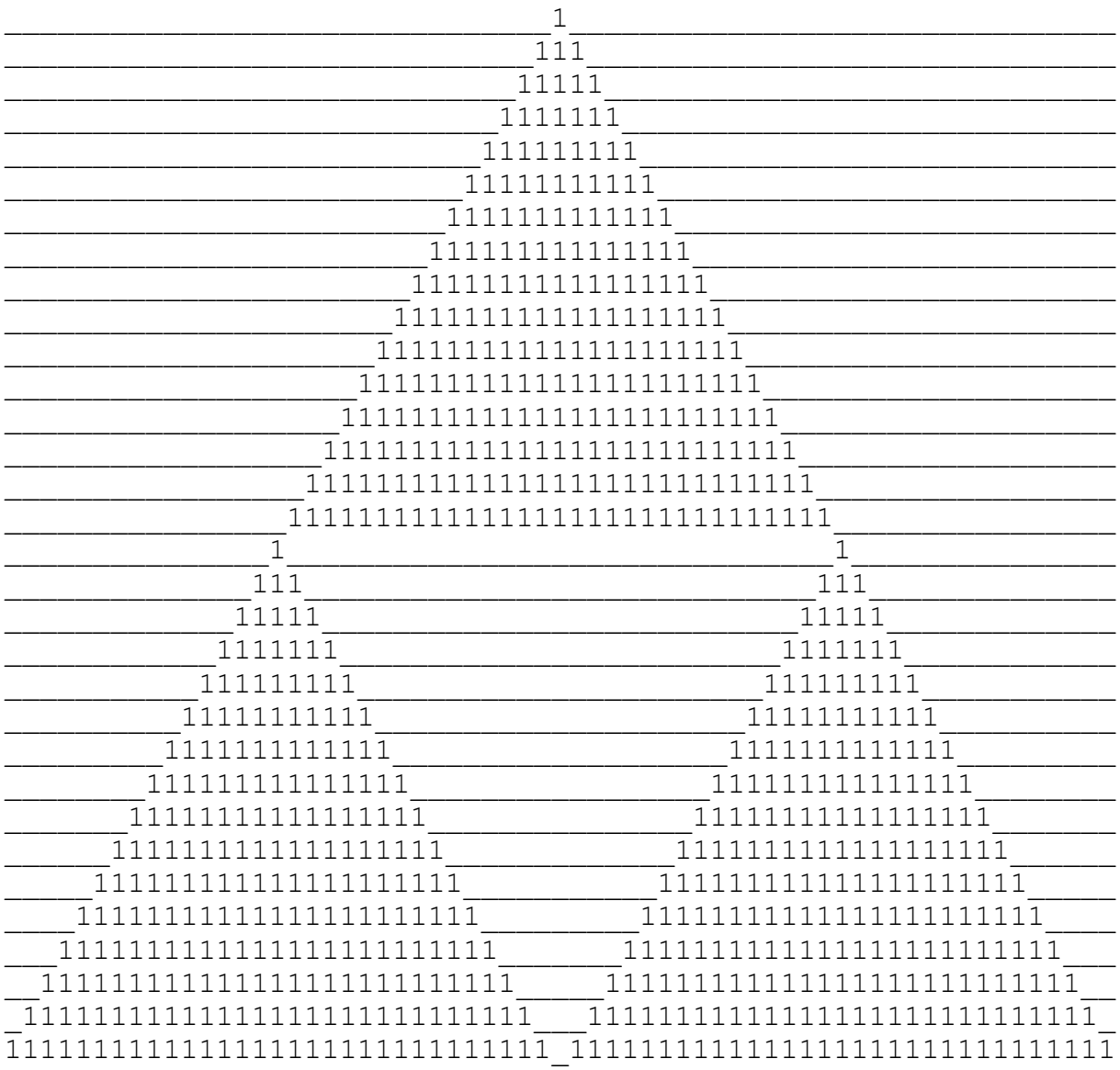
- In the first version, please implement using *ITERATION*.
- In the second version, please implement it using *RECURSION*.
    - In the recursive version, you are not allowed to use loops or any global values. Your calculations must be done only with recursive calls.
- You can put both implementations into your source code with different names.
- Do not forget to invoke both versions in the main function.
- Please assume that there might be at most 30 levels.

3. In this question, you will print identical triangles nested each other recursively. You will construct the triangles by using two digits **`'_'(underscore)`** and **`'1'`**. There will be number of iterations given as an input to your program.

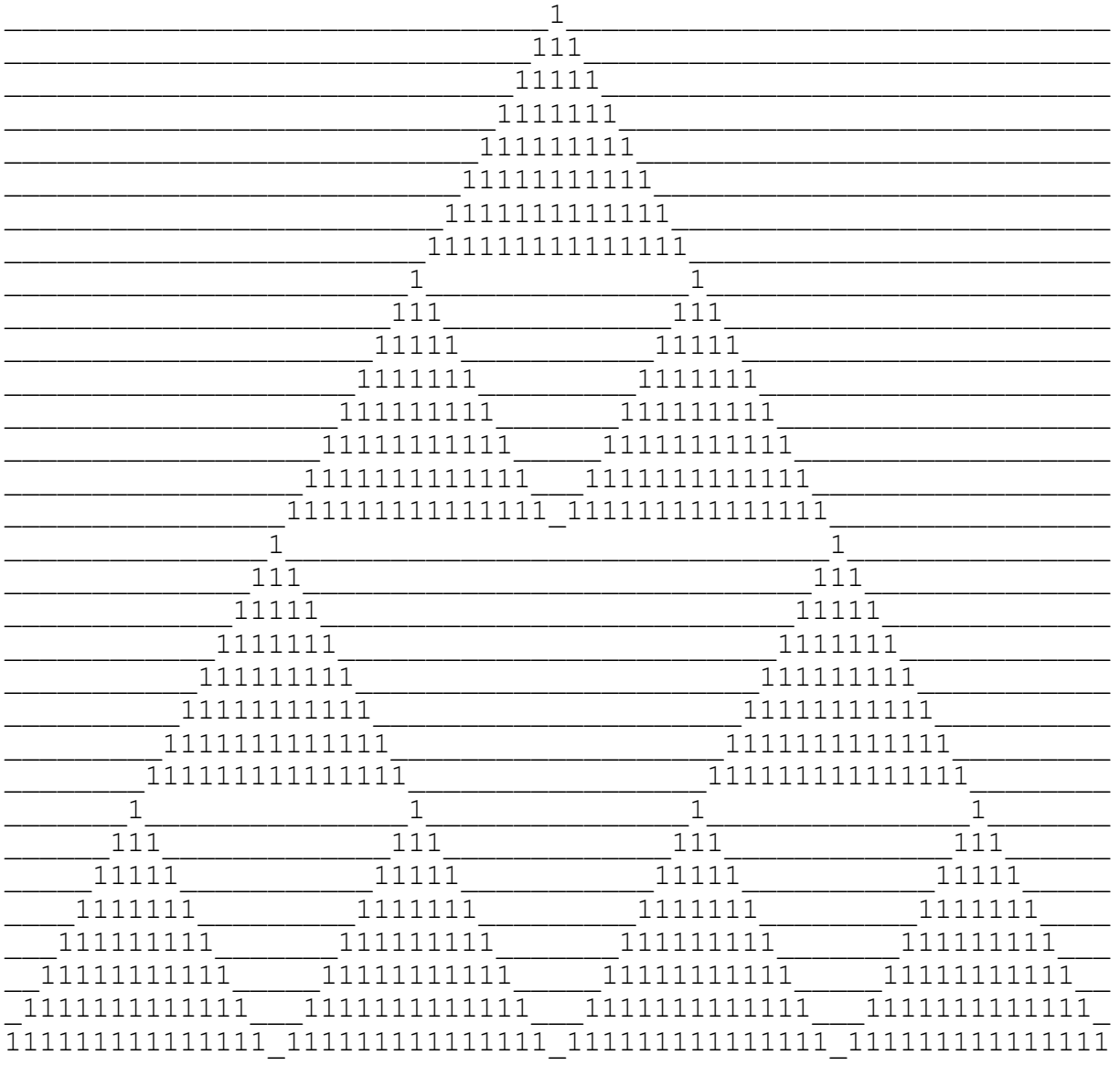
- **If the number of iterations is given as 0**, you will print a simple triangle by using 32 rows and 63 columns in a matrix as the following:



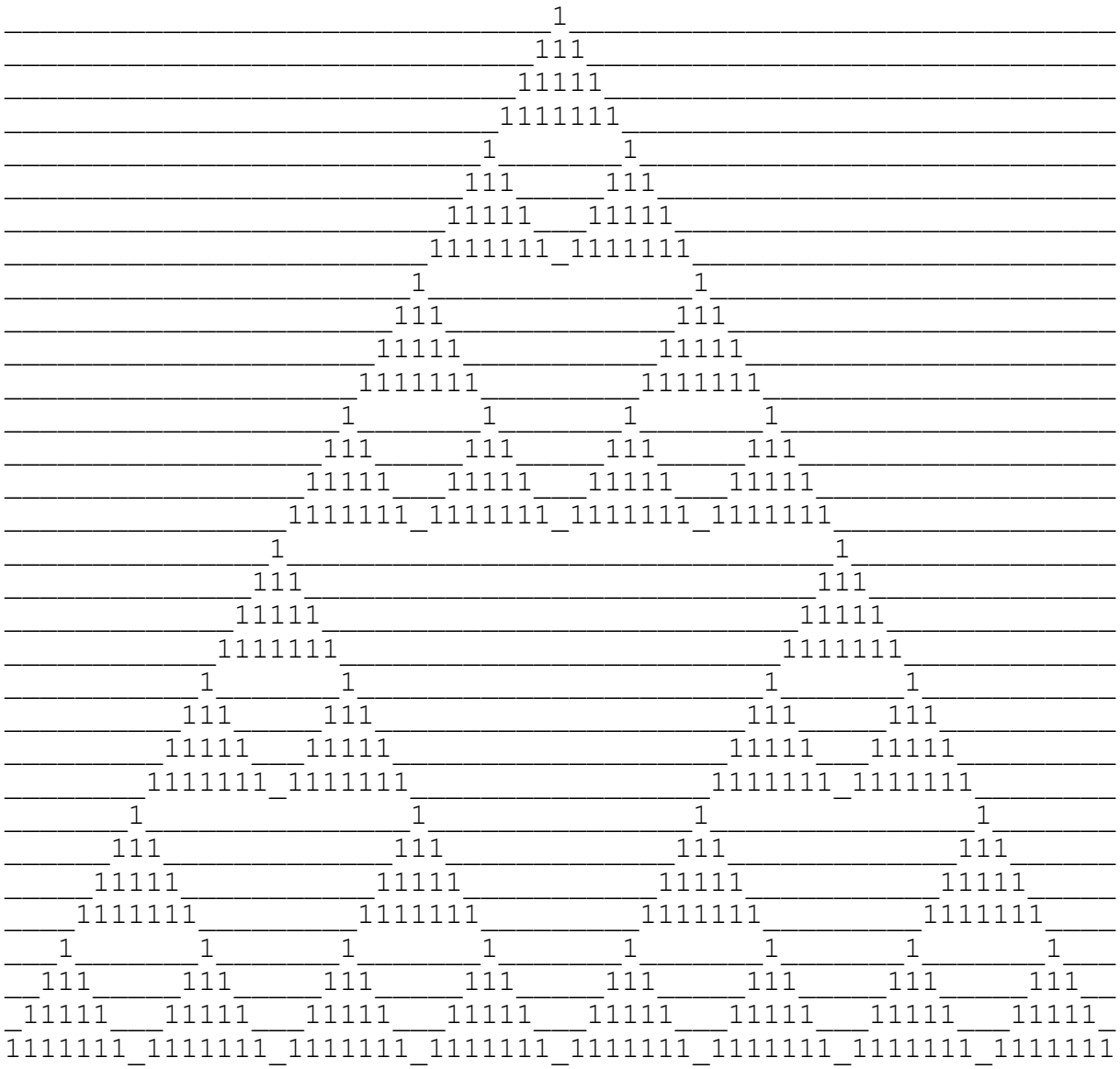
- **If the number of iterations is given as 1**, you will create 3 triangles by calculating their 3 corner points using the original triangle in the previous iteration. It should be noted that the original triangle at iteration 0 will be decomposed three identical triangles as the following:



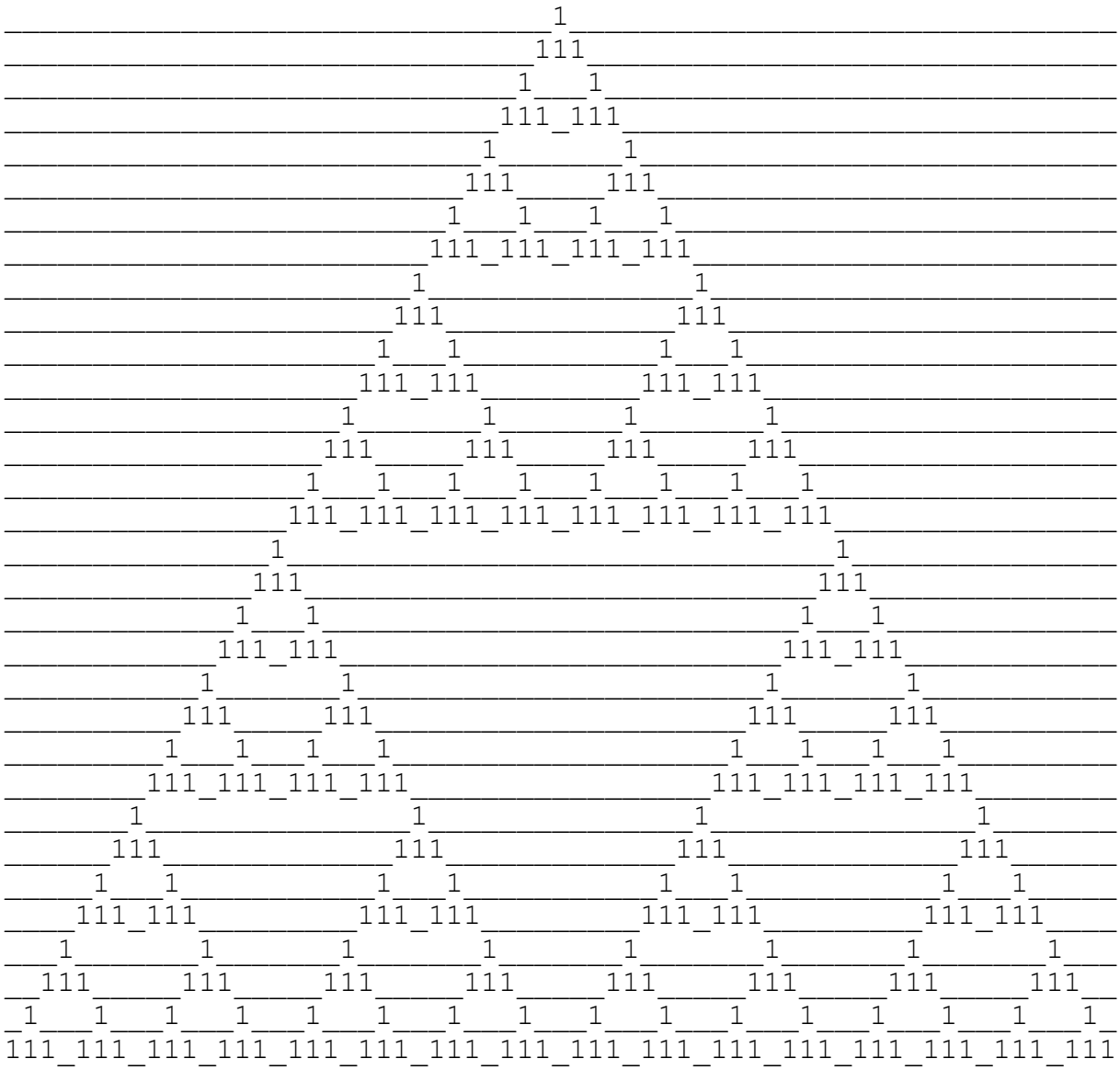
- **If the number of iterations is given as 2**, you will again create 3 triangles for each triangle in the previous iteration by calculating their 3 corner points using the triangles in the previous iteration. It should be noted that each triangle at iteration 1 will be decomposed three identical triangles as the following:



- **If the number of iterations is given as 3**, you will need to print triangles as the following:



- **If the number of iterations is given as 4**, you will need to print triangles as the following:



- The number of iterations given to your program will be less than 5.
- The output will consist of 32 rows and 63 columns, and will be composed of ones (**1**) and underscores as in the triangles above.
- Solutions using iterations will not be graded for ALL questions.
- You have to solve the problems by using RECURSION.